Guion de trabajo tutorizado

# Streaming

## Plataformas Tecnológicas

*Profesor D. José Antonio Lagares Rodríguez*
*Máster Big Data & Business Analytics (MBDA)*







Alumno: ÁLVARO RAMOS BERNABÉ

# ENUNCIADO DE LA ACTIVIDAD

*Guion oficial: «MBDA-Ejercicios Spark Streaming.pdf» (Campus Virtual UPO).*

## Objetivos

• Reforzar los conocimientos teóricos vistos en clase sobre Apache Spark Streaming, Apache Storm y Apache Flink.

• Saber elegir para cada problema de procesamiento de datos en línea la mejor herramienta de Big Data.

• Conocer e investigar el papel de las tecnologías dadas en clase en el mundo Big Data.

Ejercicio/Problemas Ejercicio 1 – 1,5 Puntos – 30 minutos Busque otros frameworks de procesamiento en tiempo real o microbatch que existan en el mercado. Elabore una tabla comparándolos con Storm, Trident, Spark y Flink.

Ejercicio 2 – 1,5 Puntos – 30 minutos De cada proveedor en la nube, liste los servicios de procesamiento en tiempo real que ofrecen.

Ejercicio 3 – 3 puntos – 60 minutos Extraiga los datos de una API en tiempo real y agrupa los resultados obtenidos. Puede usar una de las API que se proporciona en el listado (https://github.com/public-api-lists/public-api-lists) o de su elección. Puede desarrollarlo en Python usando Google Colab.

• (1,5 puntos) Si se usa Databricks u otra herramienta Cloud.

• (1,5 puntos) Si se usa una API con API_KEY.

Ejercicio 4 – 4 puntos – 180 minutos Cree un conjunto de Jobs para extraer datos desde APIs y guardarlos en Databricks. Muestre paso a paso la creación y resultado con captura de pantallas.

1) (0,50 puntos) El primer Job será bronce y servirá para extraer los datos puros. Deberá tener parámetros para poder reusar el código.

2) (0,50 puntos) El segundo Job será plata y servirá para hacer limpieza y agregaciones. De manera opcional, deberá tener parámetros para poder reusar el código.

3) (0,50 puntos) El tercer Job será oro y servirá para presentar resultados y tomar decisiones. De manera opcional, deberá tener parámetros para poder reusar el código.

4) (0,50 puntos) Organice que los Jobs se ejecuten durante varios días de manera automática creando un conjunto de datos.

5) (1 punto) Investigue y cree un pipeline ETL para orquestar los Jobs creados y organizar las dependencias entre ellos.

6) (1 punto) Cree un dashboard para mostrar los resultados de la capa oro.

Fecha y Modo de entrega La fecha de entrega está marcada dentro del Aula Virtual.

## Bibliografía

1- Página oficial de Apache Spark - http://spark.apache.org/

2- Guía de programación en Apache Spark - http://spark.apache.org/docs/latest/programming-guide.html

3- Página oficial de Apache Spark Streaming - http://spark.apache.org/streaming/

4- Guía de programación en Apache Spark Streaming –

5- http://spark.apache.org/docs/latest/streaming-programming-guide.html

6- Página oficial de Apache Storm - http://storm.apache.org/

7- Guía de programación y recursos para Apache Storm - http://storm.apache.org/documentation.html

# RESOLUCIÓN

## Ejercicio 1. Otros frameworks de procesamiento en tiempo real o micro-batch

Además de los cuatro vistos en clase —Storm, Trident, Spark Streaming y Flink—, el mercado ofrece hoy una familia amplia de motores. Para compararlos con criterio conviene fijar primero los cuatro ejes que de verdad los diferencian:

- **Modelo de ejecución:** *evento a evento* (latencia de milisegundos) frente a *micro-batch* (latencia de segundos, mayor rendimiento por registro).
- **Garantías de entrega:** *at-most-once*, *at-least-once* o *exactly-once*, y a qué coste se consiguen.
- **Gestión de estado y tiempo:** si el motor mantiene estado con puntos de control, y si distingue el *tiempo de evento* del *tiempo de proceso* para tratar datos que llegan tarde.
- **Modelo operativo:** clúster propio, biblioteca embebida en la aplicación o servicio gestionado.

| Framework | Modelo | Garantías | Estado y tiempo de evento | Rasgo distintivo |
|---|---|---|---|---|
| **Apache Storm** | Evento a evento | At-least-once (at-most-once sin *acking*) | Estado externo; sin soporte nativo de tiempo de evento | Pionero de baja latencia; hoy en desuso frente a Flink. |
| **Trident** (sobre Storm) | Micro-batch | Exactly-once | Estado gestionado por el framework | Añade transaccionalidad a Storm a costa de latencia. |
| **Spark Structured Streaming** | Micro-batch (y modo continuo experimental) | Exactly-once con *checkpointing* y sumideros idempotentes | Estado con *watermarks* y ventanas por tiempo de evento | Mismo API que el batch: un equipo que sabe Spark no aprende una herramienta nueva. |
| **Apache Flink** | Evento a evento | Exactly-once mediante *checkpoints* distribuidos (algoritmo Chandy-Lamport) | Estado nativo, *savepoints*, tiempo de evento y datos tardíos | Referencia técnica actual en streaming con estado. |
| **Kafka Streams** | Evento a evento | Exactly-once (transacciones de Kafka) | Estado local con RocksDB y *changelog* en Kafka | No es un clúster: es una biblioteca Java dentro de tu aplicación. |
| **ksqlDB** | Evento a evento | Hereda las de Kafka Streams | Ventanas y tablas materializadas | Streaming en SQL sobre Kafka, sin escribir código. |
| **Apache Beam** (Dataflow, Flink, Spark) | Unificado batch/streaming | Depende del *runner* | Modelo de ventanas y *triggers* muy completo | Abstrae el motor: el mismo código corre en varios *runners*. |
| **Apache Samza** | Evento a evento | At-least-once (exactly-once con Kafka) | Estado local con RocksDB | Nacido en LinkedIn; muy acoplado a Kafka y YARN. |
| **Apache Pulsar Functions** | Evento a evento | Effectively-once | Estado ligero | Cómputo embebido en el propio bus de mensajería. |
| **Apache Heron** | Evento a evento | At-least-once / exactly-once | Estado con *checkpoints* | Sucesor de Storm en Twitter, compatible con su API. |
| **Hazelcast Jet** | Evento a evento | Exactly-once | Estado en memoria distribuida | Motor embebible, latencias muy bajas, sin clúster externo. |
| **Bytewax** | Evento a evento | At-least-once / exactly-once (según sumidero) | Estado con *recovery* | Streaming con estado en Python puro (núcleo en Rust). |
| **Materialize / RisingWave** | Vistas materializadas incrementales | Exactly-once | Estado gestionado por el motor | Bases de datos de streaming: se consulta SQL y el resultado se mantiene siempre |

| | | | | |
|---|---|---|---|---|
| | | | | actualizado. |
| **Databricks Delta Live Tables** | Micro-batch sobre Spark | Exactly-once | Gestionado por la plataforma | Declarativo: se describe la tabla y la plataforma orquesta el flujo. |

### Lectura de la comparativa

- **Storm y Trident son hoy tecnología heredada.** Su aportación histórica fue demostrar que se podía procesar evento a evento a escala; Flink resolvió mejor el problema del estado y se quedó con ese espacio.
- **La división batch/streaming se ha difuminado.** Spark Structured Streaming y Beam ofrecen el mismo API para ambos mundos, y Flink ejecuta batch como caso particular de *streaming* acotado.
- **No siempre hace falta un clúster.** Kafka Streams, Bytewax o Hazelcast Jet son bibliotecas: el procesamiento vive dentro de la aplicación y se escala replicando instancias. Para muchos casos reales es una arquitectura más simple de operar que un clúster dedicado.
- **El SQL ha llegado al streaming.** ksqlDB, Flink SQL, Materialize y RisingWave permiten resolver en una consulta lo que antes exigía una topología de código, lo que acerca el tiempo real a perfiles analíticos.
- **Criterio de elección:** si la latencia admisible son segundos y ya se usa Spark, Structured Streaming; si hace falta estado complejo y latencia de milisegundos, Flink; si todo el ecosistema es Kafka y el volumen es moderado, Kafka Streams o ksqlDB; y si no se quiere operar infraestructura, un servicio gestionado del ejercicio 2.

## Ejercicio 2. Servicios de procesamiento en tiempo real por proveedor cloud

Los proveedores estructuran su oferta en tres piezas que conviene no confundir: **ingesta** (el bus que recibe los eventos), **procesamiento** (el motor que los transforma) y **destino analítico** (donde se consultan). Un pipeline real combina al menos una de cada.

| Proveedor | Ingesta | Procesamiento | Destino / analítica en tiempo real |
|---|---|---|---|
| **Amazon Web Services** | Kinesis Data Streams; Amazon MSK (Kafka gestionado); Data Firehose; EventBridge; IoT Core | Managed Service for Apache Flink (antes Kinesis Data Analytics); Lambda; Spark en EMR; Glue Streaming ETL | OpenSearch Service; Redshift (Streaming Ingestion); Timestream; S3 + Athena |
| **Microsoft Azure** | Event Hubs (compatible con protocolo Kafka); IoT Hub; Service Bus | Azure Stream Analytics (SQL); Functions; Spark/Flink en HDInsight; Databricks; Fabric Eventstream | Fabric Real-Time Intelligence (Eventhouse/KQL); Data Explorer; Synapse; Power BI en directo |
| **Google Cloud** | Pub/Sub; Managed Service for Apache Kafka; IoT/Eventarc | Dataflow (Apache Beam); Dataproc (Spark/Flink); Cloud Run functions; Datastream (CDC) | BigQuery (Storage Write API y suscripciones directas de Pub/Sub); Bigtable; Looker Studio |
| **IBM Cloud** | Event Streams (Kafka gestionado); MQ | Streaming Analytics; Code Engine; watsonx.data | Db2 Warehouse; Cognos Analytics |
| **Oracle Cloud** | OCI Streaming; Kafka Connect Harness | GoldenGate Stream Analytics; Functions; Data Flow (Spark) | Autonomous Data Warehouse; Analytics Cloud |
| **Alibaba Cloud** | DataHub; ApsaraMQ for Kafka | Realtime Compute for Apache Flink | Hologres; MaxCompute |
| **Plataformas independientes** | Confluent Cloud; Redpanda | Flink en Confluent Cloud; | Snowflake (Snowpipe |

| | | | |
|---|---|---|---|
| | Cloud | Redpanda Connect | Streaming, *dynamic tables*); Databricks (DLT) |

## Comparación práctica

- **El grado de gestión es el criterio decisivo.** Pub/Sub, Event Hubs y Kinesis escalan sin aprovisionar nada; MSK o un Kafka gestionado exigen dimensionar particiones y brokers, a cambio de compatibilidad total con el ecosistema Kafka y portabilidad entre nubes.
- **Interfaz de programación.** Azure Stream Analytics y ksqlDB apuestan por SQL, más accesible; Dataflow y Managed Flink exigen Beam o Flink, con mucha más potencia para estado y ventanas complejas.
- **El destino manda.** Con BigQuery o Snowflake el flujo puede terminar directamente en tabla consultable sin orquestar un motor intermedio: para muchos casos de ingesta continua, el pipeline «bus → almacén» sin transformación intermedia es más barato y simple.
- **Portabilidad.** Apache Beam y Kafka son las dos apuestas que reducen la dependencia de proveedor; los servicios propietarios (Stream Analytics, Timestream) ganan en rapidez de puesta en marcha y pierden en capacidad de migración.

## Ejercicio 3. Extracción de datos de una API en tiempo real y agrupación

Para este ejercicio se ha elegido la **API de JCDecaux** (`https://api.jcdecaux.com/vls/v1/stations`), que publica en tiempo real el estado de los sistemas públicos de bicicletas que esa empresa opera en todo el mundo, entre ellos **Sevici**, el de Sevilla. Se ha elegido esta API por dos motivos que el enunciado puntúa de forma explícita:

- **Requiere API key.** El acceso exige una clave personal obtenida en el portal de desarrolladores de JCDecaux, que viaja en el parámetro `apiKey` de la petición.
- **El trabajo se resuelve sobre una herramienta cloud.** Toda la ejecución se ha hecho en **Databricks** (edición gratuita, runtime *serverless*), con Spark y tablas Delta sobre Unity Catalog.

Un detalle importante de diseño: **la clave no está escrita en el código**. Se recibe a través de un *widget* del notebook (`dbutils.widgets.text("api_key", ...)`), que es el mismo mecanismo con el que un Job inyecta parámetros en tiempo de ejecución. Así el mismo notebook sirve para cualquier contrato de JCDecaux —Sevilla, Valencia, Bruselas— sin tocar una línea, y la credencial no queda almacenada en el fichero fuente.

### Código del ejercicio

```
import requests, datetime
from pyspark.sql.functions import col, count, sum as ssum, avg, round as sround,
when, current_timestamp

dbutils.widgets.text("contrato", "Seville", "Contrato JCDecaux (ciudad)")
dbutils.widgets.text("api_key", "", "API key de JCDecaux")
contrato = dbutils.widgets.get("contrato")
api_key  = dbutils.widgets.get("api_key")   # parametro del Job: no se escribe en
el codigo

CATALOGO, ESQUEMA = "workspace", "pt11"
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {CATALOGO}.{ESQUEMA}")

# 1. LLAMADA A LA API EN TIEMPO REAL
URL = f"https://api.jcdecaux.com/vls/v1/stations?
contract={contrato}&apiKey={api_key}"
r = requests.get(URL, timeout=30)
print("Codigo de respuesta:", r.status_code)
estaciones = r.json()
print(f"Estaciones recibidas: {len(estaciones)}")

# 2. NORMALIZACION A DATAFRAME DE SPARK Y PERSISTENCIA EN DELTA
filas = [(e["number"], e["name"], e.get("address"), e["status"],
e["bike_stands"],
          e["available_bikes"], e["available_bike_stands"],
          float(e["position"]["lat"]), float(e["position"]["lng"]),
          bool(e.get("banking")), str(e.get("last_update"))) for e in estaciones]

df = spark.createDataFrame(filas, "numero int, nombre string, direccion string,
estado string, "
                                  "anclajes int, bicis_disponibles int,
anclajes_libres int, "
```

```
                                    "lat double, lon double, con_tpv boolean,
ultima_actualizacion string")
df = df.withColumn("momento_consulta", current_timestamp())
df.write.format("delta").mode("append").saveAsTable(f"{CATALOGO}.
{ESQUEMA}.sevici_crudo")
```

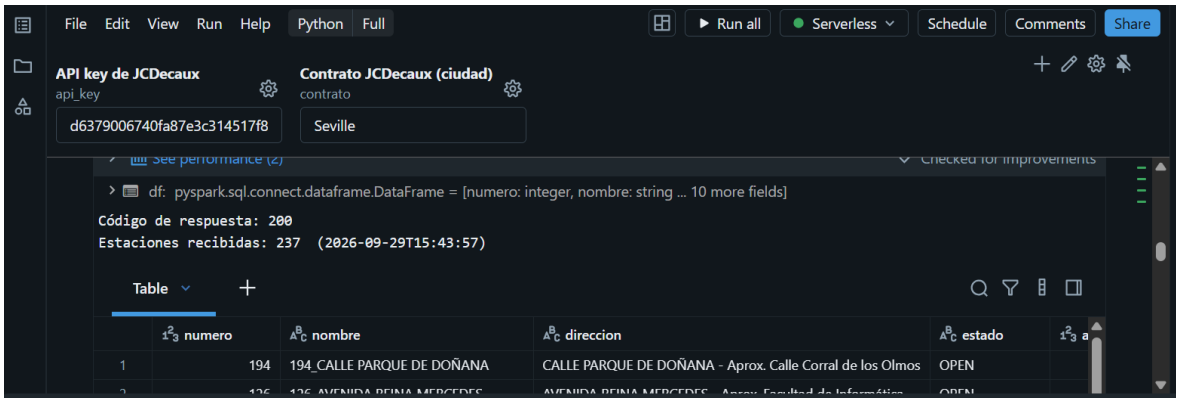



**Figura 1.** Llamada real a la API de JCDecaux desde Databricks: código de respuesta 200 y 237 estaciones recibidas. Arriba, los widgets `api_key` y `contrato` con los que se parametriza la ejecución.

### Agrupación de los resultados

La API devuelve una fila por estación: 237 registros sueltos que, por sí solos, no dicen nada del sistema. El valor aparece al agregarlos. Se han hecho **dos agrupaciones con lecturas distintas**.

**Agrupación 1 — disponibilidad por zona de la ciudad.** La zona se deriva de la latitud de cada estación (Sevilla se reparte de norte a sur a lo largo del Guadalquivir), y para cada zona se calculan estaciones, bicicletas disponibles, anclajes libres y el porcentaje de ocupación:

```
df_zona = (df.withColumn("zona",
        when(col("lat") > 37.40, "Norte").when(col("lat") < 37.37,
"Sur").otherwise("Centro"))
    .groupBy("zona")
    .agg(count("*").alias("estaciones"),
         ssum("bicis_disponibles").alias("bicis_disponibles"),
         ssum("anclajes_libres").alias("anclajes_libres"),
         ssum("anclajes").alias("anclajes_totales"),
         sround(avg("bicis_disponibles"), 1).alias("bicis_media_estacion"))
    .withColumn("ocupacion_pct", sround(col("bicis_disponibles") /
col("anclajes_totales") * 100, 1))
    .orderBy(col("ocupacion_pct").desc()))
```




**Figura 2.** Resultado de la agrupación por zona: el Centro concentra 142 de las 237 estaciones y 1.259 bicicletas disponibles.

**Agrupación 2 — estaciones en situación crítica.** La primera agregación dice cuántas bicicletas hay; esta dice dónde falla el servicio. Se clasifica cada estación según si está cerrada, sin bicicletas (nadie puede coger una) o sin anclajes libres (nadie puede dejar la suya):

```
df_critico = (df.withColumn("situacion",
        when(col("estado") != "OPEN", "Cerrada")
        .when(col("bicis_disponibles") == 0, "Sin bicis")
        .when(col("anclajes_libres") == 0, "Sin anclajes libres")
        .otherwise("Normal"))
    .groupBy("situacion").agg(count("*").alias("estaciones"))
    .orderBy(col("estaciones").desc()))
```




**Figura 3.** Estaciones por situación operativa en el instante de la consulta: 218 normales, 10 sin anclajes libres y 9 sin bicicletas.

### Lectura de los resultados

De las 237 estaciones, **19 estaban en situación crítica en ese instante** (un 8 %): 9 sin ninguna bicicleta y 10 completamente llenas. Son precisamente las que generan quejas de usuario, y el dato sale de una consulta que tarda dos segundos. Repitiendo esta misma extracción cada pocos minutos —que es exactamente lo que hace el Job del ejercicio 4— se obtiene la serie temporal que permite detectar qué estaciones se vacían siempre a la misma hora y planificar la redistribución de bicicletas con camión, que es el coste operativo principal de estos sistemas. Los agregados se guardan en la tabla Delta

`workspace.pt11.sevici_por_zona` en modo *append*, de forma que cada ejecución añade una foto más al histórico en lugar de sobrescribir la anterior.

## Ejercicio 4. Conjunto de Jobs, automatización, pipeline ETL y cuadro de mando

Este ejercicio se resuelve con una **arquitectura medallón** (bronce → plata → oro) sobre Databricks, que es el patrón estándar para este tipo de flujos: cada capa tiene una responsabilidad única y sólo lee de la anterior, de modo que un fallo se localiza en una capa concreta y se puede reprocesar sin volver a llamar a la API.

La fuente es la **API de Open-Meteo** (`api.open-meteo.com/v1/forecast`), que devuelve la observación meteorológica actual de un punto geográfico. Es una API libre, sin clave, que permite ejecutar el pipeline tantas veces como haga falta sin consumir cuota —a diferencia de la del ejercicio 3, que sí está limitada—.

| Capa | Notebook | Qué hace | Parámetros | Modo de escritura |
|---|---|---|---|---|
| **Bronce** | `PT11_bronce` | Llama a la API y guarda la observación tal cual, con metadatos de trazabilidad (momento de ingesta y origen). | `ciudad`, `latitud`, `longitud` | `append` (histórico inmutable) |
| **Plata** | `PT11_plata` | Descarta lecturas imposibles, deduplica por ciudad y hora, y agrega métricas por ciudad. | `ciudad` (`TODAS` = todas) | `overwrite` (refleja el estado actual) |
| **Oro** | `PT11_oro` | Aplica reglas de negocio: nivel de riesgo, recomendación operativa y ranking de ciudades. | `umbral_calor`, `umbral_frio`, `umbral_viento` | `append` (histórico de alertas) |

### 1. Job bronce — extracción de los datos puros, parametrizada

La parametrización se resuelve con *widgets*: el notebook declara sus parámetros, el Job los inyecta al lanzarlo y el mismo código sirve para cualquier ciudad. Ése es exactamente el requisito de «poder reusar el código».

```
dbutils.widgets.text("ciudad", "Sevilla", "Nombre de la ciudad")
dbutils.widgets.text("latitud", "37.39", "Latitud")
dbutils.widgets.text("longitud", "-5.98", "Longitud")
p_ciudad, p_lat, p_lon = (dbutils.widgets.get(k) for k in ("ciudad", "latitud",
"longitud"))

URL = (f"https://api.open-meteo.com/v1/forecast?
latitude={p_lat}&longitude={p_lon}"
       "&current_weather=true&hourly=relative_humidity_2m&timezone=Europe
%2FMadrid")
r = requests.get(URL, timeout=30)
if r.status_code != 200:
    raise Exception(f"Error en la API ({r.status_code}): {r.text[:200]}")

df = (spark.createDataFrame([(temperatura, viento, direccion, codigo, humedad,
ts_api)], esquema)
      .withColumn("ciudad", lit(p_ciudad))
      .withColumn("fecha_ingesta", current_timestamp())
      .withColumn("origen", lit("open-meteo/v1/forecast")))
```

```
# APPEND: la capa bronce es un historico inmutable, nunca se sobrescribe
df.write.format("delta").mode("append").saveAsTable("workspace.pt11.weather_bronc
e")
```

La comprobación explícita del código de respuesta no es decorativa: si la API falla, el notebook lanza una excepción y la tarea del Job queda en estado *failed*, lo que impide que las capas plata y oro se ejecuten sobre datos incompletos.

### 2. Job plata — limpieza, deduplicación y agregación

La capa plata resuelve tres problemas que siempre aparecen en una ingesta continua: valores imposibles, duplicados y falta de agregación.

```
df_limpio = (df
    .withColumn("momento", to_timestamp(col("timestamp_sensor")))
    .filter(col("temperatura").between(-40, 55))     # descarta lecturas
imposibles
    .filter(col("viento") >= 0)
    .dropDuplicates(["ciudad", "timestamp_sensor"]))  # una observacion por
ciudad y hora

df_plata = (df_limpio.groupBy("ciudad", "latitud", "longitud")
    .agg(sround(avg("temperatura"), 2).alias("temp_media"),
         smax("temperatura").alias("temp_max"),
smin("temperatura").alias("temp_min"),
         sround(stddev("temperatura"), 2).alias("temp_desv"),
         sround(avg("viento"), 2).alias("viento_medio"),
smax("viento").alias("viento_max"),
         sround(avg("humedad"), 1).alias("humedad_media"),
         count("*").alias("observaciones"),
smax("momento").alias("ultima_observacion")))
```

La deduplicación por `(ciudad, timestamp_sensor)` es la clave: Open-Meteo publica una observación por hora, así que si el Job se ejecuta dos veces dentro de la misma hora, la segunda ingesta trae la misma observación. Sin ese `dropDuplicates`, las medias quedarían sesgadas hacia las horas en las que el Job se lanzó más veces.

### 3. Job oro — resultados y toma de decisiones

La capa oro es la única que contiene **reglas de negocio**. Convierte métricas en decisiones: un nivel de riesgo, una recomendación operativa redactada y un ranking entre ciudades. Los umbrales son parámetros, de modo que el mismo código sirve para una campaña de verano (umbral de calor 32 °C) o para una obra en invierno (umbral de frío 5 °C).

```
df_oro = (df_plata
    .withColumn("nivel_riesgo",
        when(col("temp_max") > u_calor + 5, "ALTO")
        .when((col("temp_max") > u_calor) | (col("temp_min") < u_frio) |
(col("viento_max") > u_viento), "MEDIO")
        .otherwise("BAJO"))
    .withColumn("recomendacion",
        when(col("temp_max") > u_calor, "Activar protocolo de calor: refuerzo de
```

```
hidratacion y turnos")
        .when(col("temp_min") < u_frio, "Activar protocolo de frio: revision de
instalaciones expuestas")
        .when(col("viento_max") > u_viento, "Aviso por viento: asegurar elementos
en exterior")
        .otherwise("Sin accion: condiciones dentro de los umbrales"))
    .withColumn("amplitud_termica", sround(col("temp_max") - col("temp_min"),
2)))

ventana = Window.orderBy(col("temp_media").desc())
df_oro = df_oro.withColumn("ranking_calor", row_number().over(ventana))
```

Se escribe en modo `append` y con los umbrales aplicados como columnas (`umbral_calor_aplicado`, `umbral_frio_aplicado`) junto a la marca temporal de la decisión. Eso permite auditar *a posteriori* por qué se emitió una alerta un día concreto, aunque los umbrales se hayan cambiado después.

### 4. Ejecución automática durante varios días

El pipeline se ha programado como **Job de Databricks con ejecución diaria** sobre cómputo *serverless*. Al ser un Job y no una ejecución manual, Databricks lo lanza aunque el navegador esté cerrado, registra cada ejecución con su duración y su estado, y reintenta o avisa si falla.

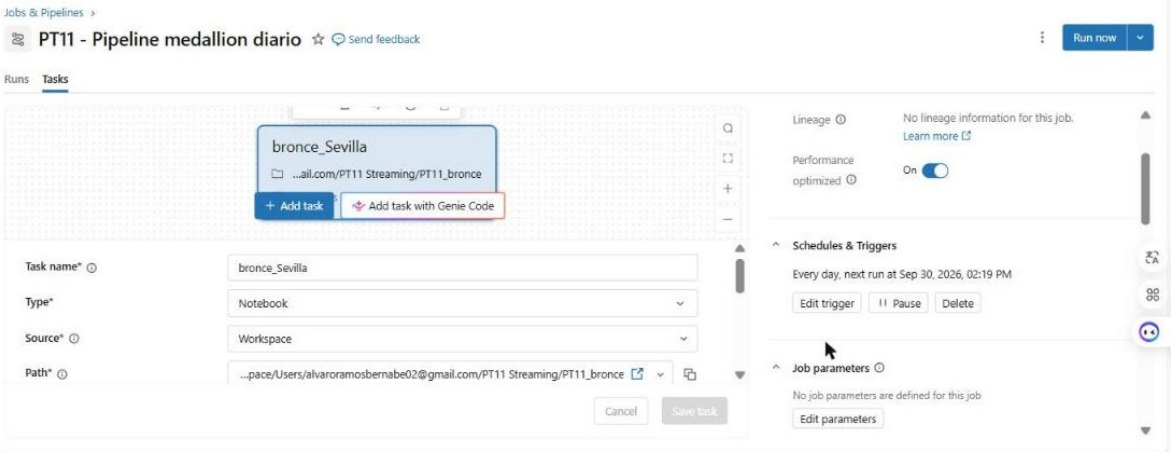



**Figura 4.** Programación del Job: ejecución automática cada día sobre cómputo serverless.

Cada ejecución diaria añade una fila por ciudad a la capa bronce. Es el propio calendario el que construye el conjunto de datos: pasados diez días hay treinta observaciones (tres ciudades × diez días) sobre las que la capa plata ya puede calcular medias y desviaciones con sentido estadístico, y la capa oro puede comparar la evolución del riesgo entre ciudades.

### 5. Pipeline ETL: orquestación y dependencias

Las tres capas no pueden ejecutarse a la vez: la plata necesita que la bronce haya terminado, y la oro necesita la plata. Esa dependencia se resuelve con un **notebook orquestador** (`PT11_pipeline`) que usa `dbutils.notebook.run()`, la primitiva de Databricks para encadenar notebooks. Cada llamada es **bloqueante**: no devuelve el control hasta que el notebook invocado termina, y si falla lanza una excepción que aborta el pipeline completo.

```
# PT11 - PIPELINE ETL: orquesta bronce -> plata -> oro con dependencias
explicitas
```

```
BASE = "/Workspace/Users/tu-cuenta@gmail.com/PT11 Streaming"

r1 = dbutils.notebook.run(BASE + "/PT11_bronce", 900, {"ciudad": "Sevilla",
"latitud": "37.39", "longitud": "-5.98"})
r2 = dbutils.notebook.run(BASE + "/PT11_bronce", 900, {"ciudad": "Madrid",
"latitud": "40.41", "longitud": "-3.70"})
r3 = dbutils.notebook.run(BASE + "/PT11_bronce", 900, {"ciudad": "Bilbao",
"latitud": "43.26", "longitud": "-2.93"})
print("BRONCE terminado para 3 ciudades")

r4 = dbutils.notebook.run(BASE + "/PT11_plata", 900, {"ciudad": "TODAS"})
print("PLATA terminada")

r5 = dbutils.notebook.run(BASE + "/PT11_oro", 900,
                          {"umbral_calor": "32.0", "umbral_frio": "5.0",
"umbral_viento": "40.0"})
print("ORO terminada. Pipeline completo: bronce -> plata -> oro")
```

Obsérvese que el mismo notebook bronce se invoca **tres veces con parámetros distintos**, una por ciudad. Ésa es la prueba práctica de que la parametrización con widgets cumple su función: un único código fuente, tres ingestas diferentes. Y el orden del script es la propia definición de dependencias: las tres ingestas terminan antes de que empiece la plata, y la plata antes que la oro.

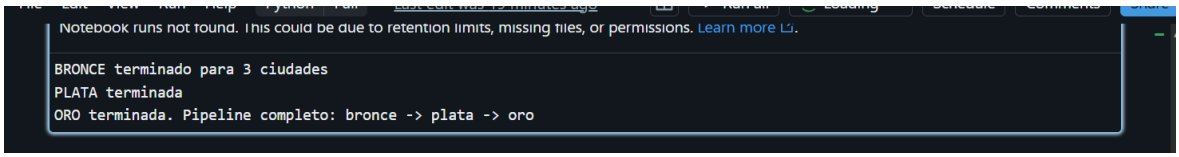



**Figura 5.** Ejecución completa del pipeline: las tres ingestas de bronce, la plata y la oro terminan en orden sin errores.

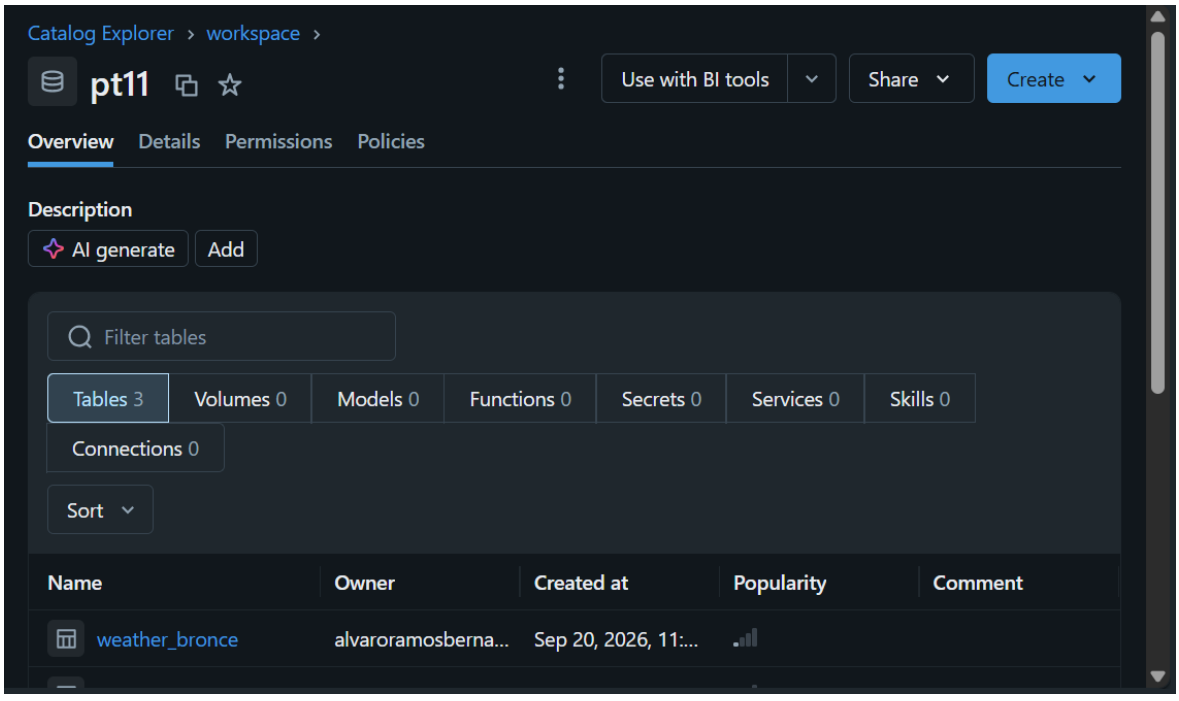



**Figura 6.** Las tres tablas Delta creadas por el pipeline en el catálogo `workspace.pt11`.

## 6. Cuadro de mando sobre la capa oro

El último requisito es presentar los resultados de la capa oro en un cuadro de mando. Se ha construido en el propio Databricks sobre la tabla `workspace.pt11.weather_oro`, de forma que se refresca solo cada vez que el Job diario escribe una nueva tanda de decisiones.




| Ciudad | Temp. Media | Temp. Máx. | Temp. Mín. | Viento Máx. | Nivel de Riesgo | Recomendación |
|---|---|---|---|---|---|---|
| Sevilla | 25,55 | 25,70 | 25,40 | 2,70 | BAJO | Sin acción: condiciones dentro de los umbrales |
| Madrid | 23,00 | 23,00 | 23,00 | 3,00 | BAJO | Sin acción: condiciones dentro de los umbrales |
| Bilbao | 20,00 | 20,00 | 20,00 | 3,20 | BAJO | Sin acción: condiciones dentro de los umbrales |

**Figura 7.** Cuadro de mando sobre `workspace.pt11.weather_oro`: tabla con la temperatura media, máxima y mínima, el viento máximo, el nivel de riesgo y la recomendación vigente de cada ciudad.

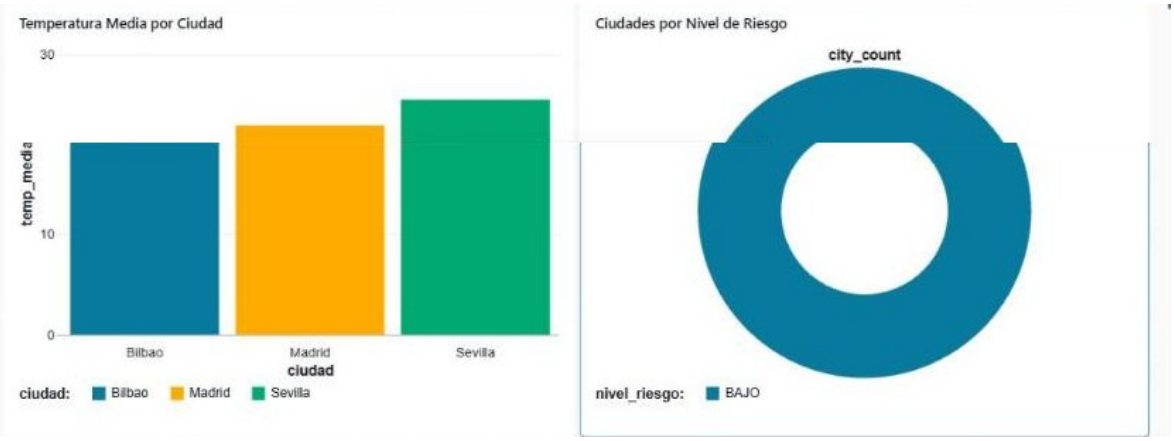



**Figura 8.** Los dos gráficos del cuadro de mando: temperatura media por ciudad y reparto de ciudades por nivel de riesgo.

## Conclusión

El ejercicio deja montado un flujo completo que funciona sin intervención: una API pública entra cada día por la capa bronce, se limpia y agrega en la plata, se convierte en decisiones en la oro y se presenta en un cuadro de mando. Las tres piezas que hacen que esto sea mantenible y no un script desechable son la **parametrización por widgets** (un código, muchas ciudades y muchos umbrales), el **modo de escritura elegido en cada capa** (append donde importa el histórico, overwrite donde importa el estado actual) y la **orquestación con dependencias explícitas**, que garantiza que ninguna capa lea datos a medio escribir.